

A Low Memory Footprint Quantized Neural Network for Depth Completion of Very Sparse Time-of-Flight Depth Maps

Xiaowen Jiang¹ Valerio Cambareri^{2*} Gianluca Agresti³
Cynthia Ifeyinwa Ugwu⁴ Adriano Simonetto⁴ Fabien Cardinaux³ Pietro Zanuttigh⁴

¹ EPFL, Switzerland ² Sony Depthsensing Solutions NV, Belgium

³ Sony Europe B.V., R&D Center, Stuttgart Laboratory 1, Germany ⁴ University of Padova, Italy

Abstract

Sparse active illumination enables precise time-of-flight depth sensing as it maximizes signal-to-noise ratio for low power budgets. However, depth completion is required to produce dense depth maps for 3D perception. We address this task with realistic illumination and sensor resolution constraints by simulating ToF datasets for indoor 3D perception with challenging sparsity levels. We propose a quantized convolutional encoder-decoder network for this task. Our model achieves optimal depth map quality by means of input pre-processing and carefully tuned training with a geometry-preserving loss function. We also achieve low memory footprint for weights and activations by means of mixed precision quantization-at-training techniques. The resulting quantized models are comparable to the state of the art in terms of quality, but they require very low GPU times and achieve up to 14-fold memory size reduction for the weights w.r.t. their floating point counterpart with minimal impact on quality metrics.

1. Introduction

Time-of-flight (ToF) sensors are active depth sensing devices [53] with the potential to provide more reliable scene understanding by true 3D perception. Due to their low power consumption and accuracy at real-time frame rates, ToF sensors were recently integrated in mobile consumer devices [33]. However, ToF relies on active illumination [53] which accounts for a relevant part of its power consumption. To use the limited power budget of a mobile device more efficiently, the scene can be illuminated with a dot pattern light source so that its radiant intensity concentrates onto a small number of regions (*dots*). A low-power ToF sensor for indoors 3D perception typically captures 500 ~ 1500 dots per frame. Because of this sparsity level, sensor fusion techniques are necessary to obtain dense

depth maps. Hereafter, we tackle this *depth completion* task by using a single RGB image frame together with a sparse depth map warped to the RGB camera view (at small baseline of a few *mm* to minimize occlusion issues w.r.t. the ToF sensor). We dub this the *sparse ToF* setting.

To obtain dense depth from a sparse depth map, deep learning-based approaches have gained significant attention, with several contributions showing competitive performances especially on autonomous driving datasets [17, 44, 47]. We instead focus on efficient indoor 3D perception with ToF. In order to attain overall low power budget for both ToF sensing and depth completion, neural networks must be designed and trained accordingly for efficient inference. In this paper we introduce a carefully designed encoder-decoder CNN for depth completion. We achieve a low memory footprint for weights and activations by means of mixed-precision quantization-aware training and show that the resulting quantized models achieve conspicuous memory size savings for weights and activations, while remaining competitive w.r.t. quality metrics. Our contributions are as follows:

- We propose an efficient depth completion model suitable for low-power sparse ToF sensing. We evaluate our model and other state-of-the-art models on two datasets: the public-domain dataset NYU-Depth v2 [43], and SDS-ST1k [6], a contributed dataset providing accurate sparse ToF and RGB camera simulations.
- We analyze the impact of input pre-processing providing fast initialization; a normals estimation block enabling supervision with ground truth normals; different loss functions and network configurations. Each of these elements contributes to achieving optimal depth map quality for the resulting model.
- We evaluate quantization-aware training strategies to quantize weights and activations of the previous model. To the best of our knowledge, mixed-precision quantization [46] has been used successfully to classification tasks. In this paper we present the first ap-

*Corresponding author: valerio.cambareri@sony.com. XJ, CU, and AS thank Sony Europe B.V. for supporting their internships.

plication of mixed precision quantization to the depth completion task. This technique allows us to minimize the weights and activations memory requirements with very limited quality degradation.

2. Related Works

2.1. Depth Completion

The main task we explore in this paper is popular in computer vision since the introduction of depth sensors [3, 11] and has been addressed by neural networks with convolutional encoder-decoder architectures [9, 24, 34, 35, 37, 39, 52]. Many other works define the state of the art in depth completion [12, 13, 20, 22, 28, 31, 40, 45]. We will compare our results against CNN solutions [8, 9, 34, 37] selected on the basis of their similar complexity (single encoder-decoder). These will be retrained on our sparse ToF datasets. There exist also higher-complexity networks leveraging, *e.g.*, multiple decoder branches [39, 51] or separate encoder branches [16, 42, 48, 52] but in this work we aimed for a more compact and lightweight model. We will only focus on *single-frame* depth completion. Many contributions extend depth completion by either self-supervision or by providing as input multiple RGB-D frames with known or inferred poses [16, 34, 38, 49, 50]. By tackling only the single-frame case, we grant temporally-independent behavior for both dynamic and static cameras and remove additional computational effort required by pose estimation via neural networks (*e.g.*, [25]) or traditional odometry. We now discuss the main works we compare against.

Sparse-to-Dense (S2D [34]) is a popular approach using a single encoder-decoder network. Its core contribution is self-supervision by pose estimation (a feature we do not leverage) as well as its long-standing role in the KITTI leaderboard compared to its low complexity. The network architecture is similar to ours. As shown in [34, Sec. 6.4] and confirmed in our experiments, its robustness to sparsity is limited. We reproduced and retrained their model.

D³ [8] is developed using DenseNet [21] layers as building blocks. Its input pre-processing is an efficient way to initialize the depth map and attain high quality while maintaining low complexity. NYU-Depth v2 results are provided, but the authors' code is not public and we could not reproduce their quality in our implementation and retraining.

Convolutional Spatial Propagation Network (CSPN [9]) was among the first works [9, 37, 51] to propose a two-stage network architecture using *spatial propagation* layers. This entails: (i) generation of a "blurred" depth map and affinity matrix; (ii) iterative refinement using the learned affinity matrix as weights for anisotropic diffusion. We retrained the authors' model via their code.



(a) 1216×352 , $K \approx 4.4\%$



(b) 304×224 , $K \approx 1.4\%$



(c) 640×480 , $K \approx 0.4\%$

Figure 1. RGB-annotation overlay of (a) KITTI (LiDAR; range: $[0, 85]m$), (b) NYU-Depth v2 (processed; range: $[0, 10]m$), (c) SDS-ST1k (sparse ToF; range: $[0, 15]m$). Annotations (red) at available sparse depth coordinates. Figure best viewed in color.

Non-Local Spatial Propagation Network (NLSPN [37]) improves upon [9], its main novelty being the use of non-local neighbors in the diffusion process, which is implemented by deformable convolutions. The authors show it achieves superior performances w.r.t. S2D and CSPN on NYU-Depth v2. In our experiments, we confirm this is indeed the most competitive approach at its network complexity. We retrained the authors' model via their code.

2.2. Neural Networks Quantization

Quantized Neural Networks (QNN) [18, 29, 54] are DNNs for which a smaller number of bits $b \ll 32$ is used to represent the weights and activations. QNNs are key to achieving efficient and low-power inference: they require considerably less memory and have a lower computational complexity, since quantized values can be stored, multiplied and accumulated efficiently.

Quantization-Aware Training (QAT) refers to a set of special algorithms to train QNNs, leading to superior results than post-training quantization. Recent QAT methods [2, 7, 10, 23, 30] proved effective on classification tasks.

These methods often use uniform quantization and the same bit width in each network layer. To improve QAT, Uhlich *et al.* [46] and Nikolić *et al.* [36] proposed to *learn* the optimal bit width for each layer while achieving a target size budget. On classification tasks, these *mixed precision* approaches show superior results since the bit widths can be allocated optimally across the network. Much less evidence of successful QAT exists for regression tasks such as depth completion.

3. Sparse ToF Datasets

We target datasets that are critically challenging in terms of sparsity. We consider indoor scenes (range: $[0, 15]m$) with rich structure and detailed objects with diverse cam-

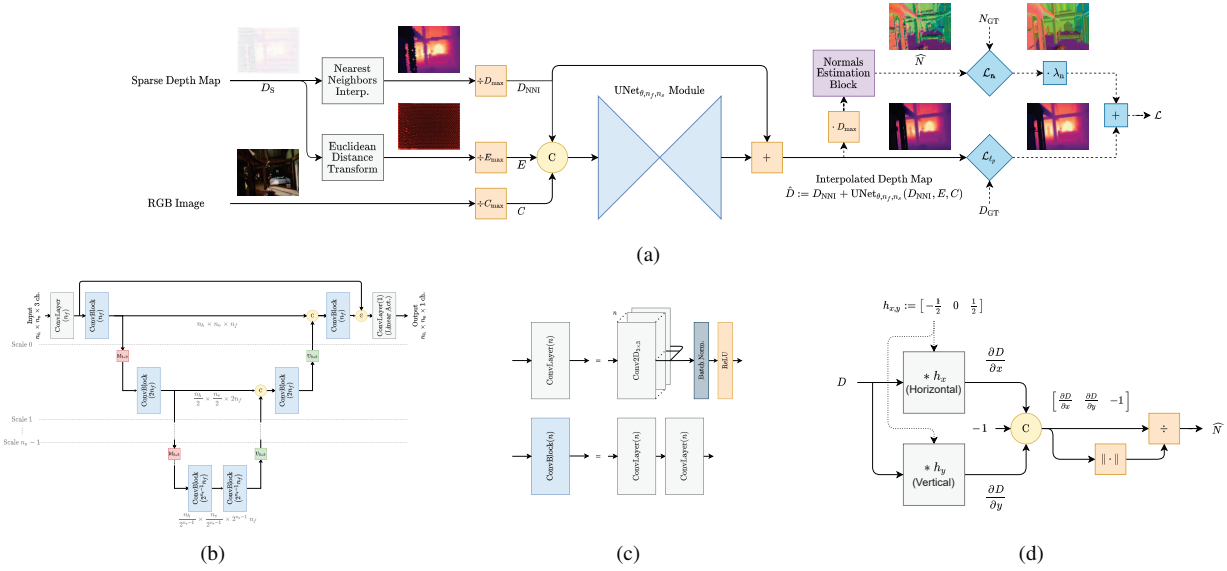


Figure 2. Summary of our network design for depth completion: (a) Network architecture. We highlight pre-processing blocks (gray); tensor operators (orange, square); training supervision operators (teal) and connections (dashed); concatenation (yellow, circle); the $UNet_{\theta, n_f, n_s}$ module (blue); the normals estimation block (purple). (b) $UNet_{\theta, n_f, n_s}$ Module. $M_{2 \times 2}$ indicates a MaxPooling2D layer with stride 2. $U_{2 \times 2}$ indicates an Upsample2D layer using nearest neighbors upsampling with factor 2. (c) Basic convolutional building blocks of the $UNet_{\theta, n_f, n_s}$ Module. (d) Normals Estimation Block, as implemented by 1-D convolution (*) in the respective directions.

era orientations. Given sparse depth maps D_S , let us define the *sparsity level* $K := 100 \frac{|\{i \in [n_h] \times [n_v] : (D_S)_i \neq \text{Invalid}\}|}{n_h n_v}$ at target depth map resolution¹ $n_h \times n_v$. We tackle very sparse settings where $K \approx 0.4 \sim 1.4\%$ (as a reference in the widely used KITTI depth completion benchmark² $K \approx 4 \sim 5\%$). A visual overview is provided in Fig. 1.

Indeed, such higher values of K simplify the depth completion task. In this work we will focus on indoor 3D perception datasets with very sparse depth maps, where traditional pipelines [27] will struggle to achieve good results.

Our main results are given on NYU-Depth v2 [43] as broadly adopted public-domain dataset. For the training set, we use a subset of $\approx 50K$ RGB-D images from the 249 standard training scenes. Each image is downsized to 320×240 then center-cropped to 304×224 , identically to [35, 37]. The main difference we introduce is that we process the depth by subsampling it with a triangular tiling dot pattern generated from sparse illumination of commercial VCSELs [33] instead of choosing random indices. This results in sparse depth maps D_S with on-average 943 active pixels ($K \approx 1.4\%$; see Fig. 1b) sampled from the full, dense depth map D_{GT} which is used as supervision. The surface normals map N_{GT} was also estimated from D_{GT} . The standard test set of 654 images is processed identically.

In addition, we provide results on SDS-ST1k [6], a contributed dataset using 8 diverse environments from Unreal

Engine [14] marketplace assets yielding $\approx 18K$ images from random camera poses at resolution 640×480 . To obtain sparse ToF data, we apply a light source with dot pattern light shading in our raytracing ToF simulator; this is received on a simulated ToF sensor compatible with low power designs and camera specifications (e.g., sensor integration time, camera intrinsics). This provides both sparse ToF depth maps D_S with realistic parallax and pattern geometry, and ground truth D_{GT} , N_{GT} . In this setting we obtain on-average 1239 active pixels, yielding a challenging $K \approx 0.4\%$ (see Fig. 1c). The recommended training/testing split yields $\approx 5K$ images as test set; this will be provided at <https://github.com/sony/ai-research-code>. We shall train our models with 160×160 non-overlapping patches (i.e., 12 per image) to meet training device memory limitations.

4. Network Design

The input of all considered depth completion methods is an RGB-D frame comprised of a color image and a sparse depth map, suitably projected to RGB camera space by assuming known intrinsics and RGB-ToF extrinsics, and with negligible RGB-ToF baseline to avoid occlusions.

4.1. Input Pre-Processing

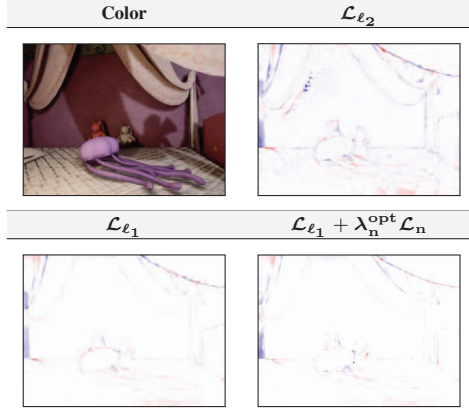
We leverage the input pre-processing method of Chen *et al.* [8]: given the sparse depth D_S , we compute an initial guess D_{NNI} of the depth map by Nearest Neighbors Interpolation (NNI), i.e., mapping each coordinate to the closest

¹Typically equal to the input RGB frame resolution or a cropping of it.

²By computing K over the standard validation set of KITTI Depth Completion.

$\mathcal{L}$	λ_n	RMSE (mm)	MNS
$\mathcal{L}_{\ell_2}$	n/a	105.2	0.538
$\mathcal{L}_{\ell_1}$	n/a	97.8	0.754
$\mathcal{L}_{\ell_1} + \lambda_n \mathcal{L}_n$	1	259.0	0.864
$\mathcal{L}_{\ell_1} + \lambda_n \mathcal{L}_n$	10^{-1}	227.0	0.868
$\mathcal{L}_{\ell_1} + \lambda_n \mathcal{L}_n$	10^{-2}	101.5	0.838
$\mathcal{L}_{\ell_1} + \lambda_n \mathcal{L}_n$	10^{-3}	98.7	0.783
$\mathcal{L}_{\ell_1} + \lambda_n \mathcal{L}_n$	10^{-4}	109.3	0.764

(a)



(b)

Figure 3. Loss function tuning on SDS-ST1k (resolution 640×480): (a) Quantitative analysis; (b) Qualitative analysis of error maps (range: $[-500, 500]$ mm). Figure best viewed in color.

sparse depth pixel in the Euclidean sense. We also compute the Euclidean Distance Transform (EDT) E with [15] (via OpenCV) given the 2-D valid coordinates of D_S . NNI is obtained by the same call, which takes median CPU time $t_{\text{NNI}} = 4.99$ ms over SDS-ST1k and $t_{\text{NNI}} = 0.99$ ms over NYU-Depth v2. We normalize D_{NNI} by the maximum range $D_{\text{max}} := 15m$ and E by $E_{\text{max}} := 40$, which we find empirically to be an appropriate normalization value for our datasets. The EDT feeds the CNN an uncertainty map of D_{NNI} , i.e., it is low where sparse depth is known and progressively higher when it is not. We tried replacing NNI with linear interpolation³ in preliminary tests, but this did not show any benefits while increasing CPU time.

4.2. Network Architecture

A summary of our supervised approach is given in Fig. 2. The core CNN element (Fig. 2b) is a simple “UNet-like” [41] template, dubbed $\text{UNet}_{\theta, n_f, n_s}$, of which we tune the number of scales n_s and feature maps n_f relative to the highest scale. Our reference is $n_s = 5, n_f = 64$. The number of feature maps doubles at each scale as their spatial resolution is reduced, up to 2^{n_s-1} at the lowest scale. The basic building blocks (Fig. 2c) are standard 2-D con-

³Linear interpolation on a non-uniform grid due to the dot pattern requires Delaunay triangulation, which is costly and cannot be reused as the pattern can change at every frame.

volutions with 3×3 filters, with padding to same resolution as the layer input and stride 1. We chose to replace transpose convolutions in the decoder with upsampling and convolution layers, as preliminary tests indicated better performance on our datasets. All skip connections are by concatenation except for the last one, which is a tensor addition w.r.t. D_{NNI} . Thus, we have the (normalized) interpolated depth $\hat{D} := D_{\text{NNI}} + \text{UNet}_{\theta, n_f, n_s}(D_{\text{NNI}}, E, C)$, i.e., $\text{UNet}_{\theta, n_f, n_s}$ computes a residual w.r.t. the initial guess.

4.3. Normals Estimation Block

As shown in Fig. 2a, we estimate the surface normals map $\hat{N}$ from the interpolated depth map $\hat{D}$ rather than using costly dedicated decoders [39]. We resort to a well-known approximation that is directly applicable to depth maps; the computation graph is reported in Fig. 2d. Given a depth map tensor D at the input, we approximate horizontal and vertical image axes derivatives by centered differences using two fixed-kernel 1-D convolutions yielding the tensors $(\frac{\partial D}{\partial x}, \frac{\partial D}{\partial y})$. We then have at the i -th pixel the normal vector

$$(\hat{N})_i := \frac{\begin{bmatrix} (\frac{\partial D}{\partial x})_i & (\frac{\partial D}{\partial y})_i & -1 \end{bmatrix}}{\sqrt{(\frac{\partial D}{\partial x})_i^2 + (\frac{\partial D}{\partial y})_i^2 + 1}}. \quad (1)$$

We found this small-kernel method sufficiently accurate to enforce similarity⁴ between $(\hat{N})_i, (N_{\text{GT}})_i$ at training.

4.4. Loss Function

Our supervised training follows the flow in Fig. 2a. The supervision is comprised of two terms. The first term measures the ℓ_p -norm distance between (normalized) ground truth D_{GT} and $\hat{D}$, i.e., for dataset $\mathcal{D}$ with J samples,

$$\mathcal{L}_p(\mathcal{D}) := \frac{1}{J} \sum_{j=1}^J \|\text{vec}(\hat{D}_j - D_{\text{GT},j})\|_p^p, \quad p = 1, 2. \quad (2)$$

The second term measures similarity between the dense ground truth normals map N_{GT} and $\hat{N}$ from the block in Fig. 2d: for each i -th pixel normal we measure the cosine similarity $\cos \alpha = \langle (N_{\text{GT}})_i, (\hat{N})_i \rangle$. We then take its average over the dataset and define the normals loss

$$\mathcal{L}_n(\mathcal{D}) := -\frac{1}{J n_h n_v} \sum_{j=1}^J \sum_{i=1}^{n_h n_v} \langle (N_{\text{GT},j})_i, (\hat{N}_j)_i \rangle \quad (3)$$

with range $[-1, 1]$ (at -1 all normals are identical). In the presence of invalid ground truth depth or normals map (e.g., for invalid pixels in NYU-Depth v2) we exclude the corresponding pixel from the loss computations.

Only part of the literature uses normals supervision⁵ [39, 51], while all depth completion approaches use depth supervision providing scale. Normals supervision alone is not

⁴We ensure the surface normals’ orientation is always consistently pointing towards the camera both in N_{GT} and $\hat{N}$.

⁵Smoothness losses on the gradients of $\hat{D}$ are also common [34, 42] but not considered here as the normals loss implements similar behavior in a supervised and more geometrically sound fashion.

n_f	n_s	MParams	RMSE (mm)	MNS
64	5	50.3	98.7	0.783
32	5	12.6	105.0	0.775
16	5	3.2	181.1	0.751
64	4	12.6	219.2	0.779
64	3	3.1	248.4	0.761
32	4	3.1	274.8	0.767

Table 1. Network tuning on SDS-ST1k (resolution 640×480). We confirm a steady decrease of quality metrics as the feature maps n_f relative to the highest scale and the scales n_s are reduced.

sufficient to retrieve scale. Thus, we minimize

$$\mathcal{L}(\mathcal{D}) := \mathcal{L}_p(\mathcal{D}) + \lambda_n \mathcal{L}_n(\mathcal{D}) \quad (4)$$

which balances between scale-dependent and scale-independent terms using $\lambda_n \in [0, 1]$. This weight depends on the dataset and must be tuned empirically. To do so, we run preliminary tests of our approach (Fig. 2a) on SDS-ST1k under different⁶ $\mathcal{L} := \{\mathcal{L}_{\ell_2}, \mathcal{L}_{\ell_1}, \mathcal{L}_{\ell_1} + \lambda_n \mathcal{L}_n\}$ and $\lambda_n := 10^{-q}$, $q = 0, \dots, 4$, in the same settings described in Sec. 6.1. Indeed, defining the conventional RMSE and Mean Normals Similarity

$$\text{MNS} := \frac{1}{J n_h n_v} \sum_{j=1}^J \sum_{i=1}^{n_h n_v} \langle (N_{\text{GT},j})_i, (\hat{N}_j)_i \rangle \quad (5)$$

as quality metrics we see that large values of λ_n degrade the RMSE, and *vice versa* for MNS. We find the best trade-off at $\lambda_n^{\text{opt}} = 10^{-3}$. As confirmation of this, in Fig. 3b we report one sample error map $\hat{D} - D_{\text{GT}}$ under different $\mathcal{L}$. As we move from $\mathcal{L}_{\ell_2}$ to $\mathcal{L}_{\ell_1}$ the observed error map pattern artefact disappears. Remaining errors at depth map discontinuities are reduced by using the normals loss $\mathcal{L}_n$.

4.5. Network Tuning

We investigated simple strategies to reduce the size of the initial model, such as tuning the $\text{UNet}_{\theta, n_f, n_s}$ module by its number of feature maps n_f relative to the highest scale, and the number of scales n_s . Our preliminary tests, in the same setting as Sec. 6.1 on SDS-ST1k, actually confirm (Tab. 1) that any pair smaller than $(n_f, n_s) = (64, 5)$ leads to a rapid degradation of RMSE and MNS, with the reduction of n_s (shallower network) being generally worse than that of n_f . Thus, we choose to achieve memory size reduction by quantization of the full model with $(n_f, n_s) = (64, 5)$ which, as we will show, allows for a more compact model with limited impact on quality metrics.

5. Quantization-Aware Training

Let $Q(x; \phi)$ be an arbitrary scalar quantizer with parameters ϕ , which maps $x \in \mathbb{R}$ to discrete values $\{q_1, \dots, q_I\}$ represented by b bits, $b : I \leq 2^b$. This quantizer may

⁶We verified in preliminary tests that $\mathcal{L}_{\ell_2} + \lambda_n \mathcal{L}_n$ is systematically worse than the ℓ_1 case and excluded it accordingly.

be applied to both weights and activations to reduce their memory size requirements. This can be done by either post-training quantization or by Quantization-Aware Training (QAT), which generally achieves better accuracy than the former. QAT consists in training the quantized version of a DNN (in this case, CNN) while applying $Q(x; \phi)$ to its weights and/or activations. However, this is challenging as the gradient through the quantizer is zero almost everywhere; a solution to this is the Straight-Through Estimator (STE) [4]. A *symmetric uniform* quantizer $Q_U(x; \phi)$ then maps $x \in \mathbb{R}$ to uniformly quantized values by

$$q = Q_U(x; \phi) := \text{sign}(x) \begin{cases} d \left\lfloor \frac{|x|}{d} + \frac{1}{2} \right\rfloor & |x| \leq q_{\max} \\ q_{\max} & |x| > q_{\max} \end{cases}, \quad (6)$$

where the parameter vector $\phi := [d, q_{\max}, b]^T$ with step size $d \in \mathbb{R}^+$, maximum value $q_{\max} \in \mathbb{R}^+$, and number of bits or *bit width* $b \in \mathbb{N} : b \geq 2$ of the quantized value q .

5.1. Uniform Precision Models

We start by considering *uniform precision* (UP) QAT, *i.e.*, every weight (or activation) of every layer has the same predefined bit width b and QAT optimizes over the remaining parameters $d, q_{\max}$ of ϕ . We conducted extensive experiments to preserve depth completion accuracy while quantizing with (6) the weights and activations in our model. The most effective UP QAT training procedure was obtained by initializing from the pretrained float32 model and using learnable $q_{\max}$ as in the Trained Quantization Threshold (TQT) approach [23]; to stabilize the QAT, we used cosine learning rate decay scheduling [32], and we first trained with RMSprop until convergence (32 epochs in our case), followed by Adam [26] for 20 more epochs.

5.2. Mixed Precision Models

Recent results [36, 46] show that, for a given memory size budget, better accuracy than UP QAT can be achieved using *mixed precision* (MP) QAT, *i.e.*, each layer uses its own bit width *learned at training* to fit a target total network size. Following [46] we parametrize $Q_U(x; \phi)$ with their range and step size d , from which the bit width $b := \lceil 1 + \log_2(q_{\max}/d + 1) \rceil$ is then inferred. With this parametrization and STE [4] gradient, we can use standard stochastic gradient descent methods to learn per-layer optimal bit widths jointly with network parameters. To achieve target network sizes we add to (4) *size constraints* on the weights and activations as penalty terms [5], minimizing

$$\mathcal{L}_{\text{MP}} = \mathcal{L}(\mathcal{D}) + \lambda_W \max(0, S_W)^2 + \lambda_A \max(0, S_A)^2 \quad (7)$$

$$S_x = (\sum_{l=1}^L S_x^l) - S_x^o, S_x^l = N_{x,l} b_{x,l}, S_x^o = \bar{b}_x \sum_{l=1}^L N_{x,l} \quad (8)$$

Precision	Weights Size (MB)	RMSE (mm)	MNS
float32	198.5	98.7	0.783
Uniform (W_4^{UP})	25.1	100.3	0.762
Mixed (W_*)	13.9	94.5	0.783

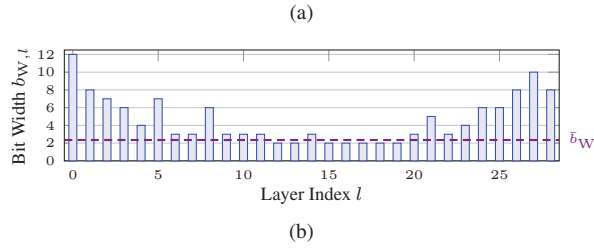


Figure 4. Weights quantization. (a) Results on SDS-ST1k (resolution 640×480) for the most compact UP model W_4^{UP} at bit width $b = 4$ and MP model W_* at average bit width $\bar{b}_W = 2.35$. (b) Per-layer precision allocation of W_* (last float32 layer omitted) and the corresponding $\bar{b}_W$ (dashed).

where $x := \{W, A\}$ denote weights or activations; S_x^l their per-layer total memory size for $N_{x,l}$ coefficients with learned bit widths $b_{x,l}$; S_x^o is the *target size* which we tune⁷ by a reference *average bit width* $\bar{b}_x$. The parameters $\lambda_W \in \mathbb{R}^+$ and $\lambda_A \in \mathbb{R}^+$ are chosen to balance the respective penalties. We empirically set them following the criteria in [46, Sec. 4], yielding $\lambda_W \approx 2.66 \cdot 10^{-7}$, $\lambda_A \approx 1.73 \cdot 10^{-6}$. In MP QAT we also initialize network parameters from the pretrained float32 model, and run Adam for 60 epochs until convergence with cosine learning rate decay scheduling.

5.3. Weights Quantization

We initially compare *weights-only* UP QAT and MP QAT against the pretrained float32 model, with the same dataset and setting as Sec. 4.5. In all our models the last layer is at float32 precision as it directly produces the regression output in Fig. 2a. We refer to UP models W_b^{UP} and MP models W_b at (fixed or average) bit width b . We report the most compact UP model W_4^{UP} ($b = 4$) providing best quality metrics, and compare it against the most compact MP model W_* , where $*$ denotes target size $S_{W,*}^o := 14$ MB, i.e., $\bar{b}_W \approx 2.35$ bit when setting the size constraint.

The results in Fig. 4a allow us to establish that MP QAT is superior to UP, as with half of the weights memory size we see no degradation of MNS and a slight improvement in RMSE due to refinement from the pretrained float32 model. We also observe in Fig. 4b how the per-layer precision allocation of MP naturally tends to assign smaller (larger) bit widths to inner (outer) layers of the QNN.

5.4. Activations Quantization

For efficient inference on mobile or low-power devices, it is crucial to consider the quantization of both weights

⁷This tuning can arbitrarily refer to an average bit width or to a total size (in which case the average as defined in (8) can be fractional).

and activations. Indeed, activations can dominate memory requirements at inference (especially for encoder-decoder networks such as ours) and may additionally cost many read/write accesses of buffer memory, with large impact on energy consumption. As we observed that MP is capable of achieving superior performances than UP for the weights-only case, our final results (Sec. 6.3) will consider MP QAT with weights and activations quantization at 4 to 8 bits; we therefore denote them by $W_{b'}A_{b''}$ where $b', b'' \geq 2$ are average bit widths that set the respective size constraints.

6. Results

We present our results using the same quality metrics of most prior works [8, 35]: RMSE, Mean Absolute Error (MAE), Mean Absolute Relative Error (MRE), and $\delta_i, i = 1, 2, 3$; we also add the MNS metric (5) to study normals similarity. Where reported, the median GPU time per prediction (t_{GPU}) is measured on an NVidia GTX 1080 Ti. The reported results are given on full-frame test sets from Sec. 3.

6.1. Training Setup

We train our depth completion model for 40 epochs until convergence using RMSprop as the optimizer with learning rate $\rho := 10^{-4}$ and batch size of 8 (SDS-ST1k) or 4 (NYU-Depth v2). Given its very large resolution, all trainings of our model on SDS-ST1k leverage patches of 160×160 , but we switch to full 640×480 inference when computing quality metrics since our networks are fully convolutional.

We also retrain other competing methods for comparison, by following the supervised training procedures described by the respective authors as reproducible from their codes (S2D, CSPN and NLSPN) or descriptions (D³), while providing as input our sparse ToF datasets. We also tested the authors' CSPN and NLSPN models pretrained on NYU-Depth v2 with randomly sampled patterns, but the resulting performances were worse than our retraining.

In addition to the float32 model, we consider our quantized MP models W_8A_8, W_4A_8, W_4A_4 (Sec. 5.4) and W_* (Sec. 5.3)⁸; the subscripts denote the respective average bit widths. For MP QAT, we use the NNabla [19] implementation of [46]. In this case we obtained the best results by training our MP models starting from the pretrained float32 model, with the Adam optimizer for 60 epochs until convergence, starting from learning rate $\rho := 10^{-4}$ with cosine learning rate decay scheduling [32].

6.2. Depth Completion Models

We first discuss some quantitative results of our method (in float32 precision) against others in Tab. 2. We achieve very good results on both evaluated datasets despite their different sparsity level K : our method ranks second-best on

⁸In W_* activations are left float32.

Method	Weights Size (MB)	Activations Size (MB)	RMSE (mm)	MAE (mm)	MRE (%)	MNS	δ_1 (%)	δ_2 (%)	δ_3 (%)	Median t_{GPU} (ms)
NYU-Depth v2 (center cropped, 304×224)										
S2D	90.1	322.7	72.1	34.1	1.230	0.538	99.79	99.97	99.99	4.83 (0)
D ³	7.8	59.9	128.3	46.9	1.760	0.422	99.13	99.79	99.94	13.78 (0.99 ^{CPU})
CSPN	832.1	44.2	79.0	28.5	0.970	0.752	99.68	99.95	99.99	63.97 (14.02)
NLSPN	100.1	200.2	61.4	22.5	0.750	0.800	99.83	99.98	100.00	42.50 (7.93)
Ours	198.5	145.5	<u>63.7</u>	<u>23.2</u>	<u>0.782</u>	<u>0.790</u>	<u>99.81</u>	<u>99.98</u>	<u>99.99</u>	<u>5.11</u> (0.99 ^{CPU})
SDS-ST1k (640 × 480)										
S2D	90.1	1455.5	643.3	589.9	29.130	0.158	55.45	83.27	94.25	6.02 (0)
D ³	7.8	270.2	176.7	57.4	2.500	0.572	98.40	99.40	99.70	18.83 (4.99 ^{CPU})
CSPN	832.1	199.2	174.3	97.1	3.850	0.687	98.60	99.60	99.78	154.62 (23.50)
NLSPN	100.1	890.6	<u>99.1</u>	<u>29.4</u>	<u>1.150</u>	<u>0.775</u>	<u>99.39</u>	99.81	99.92	148.68 (11.18)
Ours	198.5	656.3	98.7	21.7	0.829	0.783	99.45	<u>99.81</u>	<u>99.91</u>	<u>10.30</u> (4.99 ^{CPU})

Table 2. Depth completion models. The median t_{GPU} is given as $t_{\text{total}}(t_{\text{initialization}})$. In ours and D³, initialization is computed on ^{CPU}.

Precision	Weights Size (MB)	Activations Size (MB)	RMSE (mm)	MAE (mm)	MRE (%)	MNS	δ_1 (%)	δ_2 (%)	δ_3 (%)
NYU-Depth v2 (center cropped, 304×224)									
float32	198.5	145.5	63.7	23.2	0.782	0.790	99.81	99.98	99.99
W ₈ A ₈	22.1	47.9	64.2	23.4	0.784	0.793	99.81	99.98	99.99
W ₄ A ₈	<u>22.0</u>	<u>36.2</u>	<u>64.3</u>	<u>23.5</u>	<u>0.793</u>	<u>0.790</u>	<u>99.81</u>	<u>99.98</u>	<u>99.99</u>
W ₄ A ₄	23.7	17.6	70.3	26.1	0.894	0.739	99.76	99.97	99.95
W _*	13.7	145.5	64.9	23.6	0.796	0.786	99.81	99.98	99.99
SDS-ST1k (640 × 480)									
float32	198.5	656.3	98.7	21.7	0.829	0.783	99.45	99.81	99.91
W ₈ A ₈	45.9	164.1	101.7	22.5	0.874	<u>0.781</u>	99.40	99.79	99.91
W ₄ A ₈	<u>20.7</u>	<u>163.3</u>	<u>101.3</u>	<u>21.1</u>	<u>0.810</u>	<u>0.776</u>	<u>99.48</u>	<u>99.83</u>	<u>99.92</u>
W ₄ A ₄	21.1	78.4	99.0	24.4	0.958	0.638	99.41	99.81	99.92
W _*	13.9	656.3	94.5	20.8	0.808	0.784	99.48	99.83	99.92

Table 3. Quantized network models. Among MP QNN models, we highlight the best trade-off between lowest memory footprint and best quality metrics for weights and activations (yellow) and weights-only (orange). Figure best viewed in color.

NYU-Depth v2 with a RMSE of 63.7 mm and **best** on SDS-ST1k with a RMSE of 98.7 mm. MAE and MNS confirm the RMSE ranking. We also achieve very competitive t_{GPU} by virtue of the simple initialization obtained via NNI and simple encoder-decoder architecture of Fig. 2a.

NLSPN provides the most competitive performances: it is quality-wise the **best** approach on NYU-Depth v2 (but the RMSE gap w.r.t. ours is very limited, only 2.3 mm) and **second-best** on SDS-ST1k, as confirmed by all quality metrics as well as visual results (Sec. 6.4). However, its t_{GPU} is relatively high even if the model is substantially compact. This is due to spatial propagation, which requires several iterations (18 as recommended in [37]) that linearly increase t_{GPU} even at constant memory footprint. t_{GPU} also notably increases significantly between 304×224 and 640×480 resolutions. Moreover, the initialization cost (*i.e.*, the input encoder-decoder module of NLSPN) is substantially higher than that of our method, which runs on CPU.

As for the other methods, it is possible to see that CSPN has slightly but consistently inferior performances than NLSPN and ours on both datasets. We also observed that

S2D, even if very fast, yields sub-optimal quality metrics on NYU-Depth v2, and poor quality metrics on SDS-ST1k given its larger resolution at lower K . This confirms the findings in [34, Sec. 6.4] that the RMSE of S2D deteriorates for lower sparsity level K and that structured subsampling patterns yield worse performances than random ones. Finally, we observed that D³, even if starting from the same initialization as ours, yields worse RMSE and MAE.

6.3. Quantized Network Models

The QAT results for our MP models are given in Tab. 3. Among those models, we highlight our best weights-only MP QAT result for W_{*}; the weights size constraint of 14 MB is met by weights memory size of 13.9 MB after training. This yields 14-fold memory size reduction w.r.t. the float32 model at 198.5 MB. There, we observe limited and graceful degradation of the quality metrics⁹ (*e.g.*, the RMSE loss is only 1.2 mm on NYU-Depth v2) when reduc-

⁹We do not report t_{GPU} as comparable with that of the float32 model and not indicative of computation time on dedicated hardware, since fixed-precision operations are here emulated by float32 operations as in [36,46].

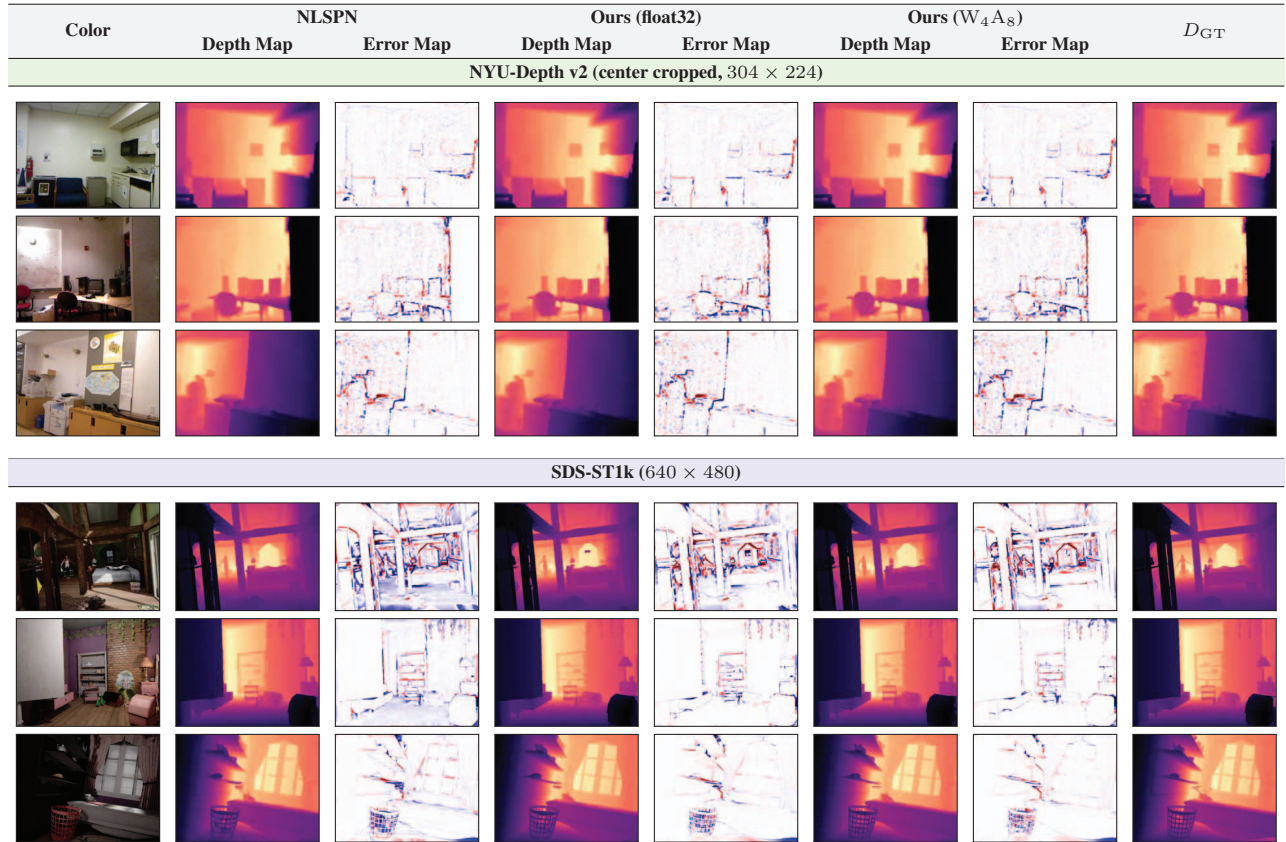


Figure 5. Qualitative results. We report, for three arbitrary frames in the test sets of NYU-Depth v2 (top rows) and SDS-ST1k (bottom rows), the predicted depth maps $\hat{D}$ and error maps $\hat{D} - D_{GT}$ (range: $[-500, 500]mm$). Figure best viewed in color.

ing precision of the weights and activations in the float32 model. However, as activations are also crucial for reducing memory footprint (Sec. 5.4), we remark that the overall most compact model with minimal impact on quality metrics is W_4A_8 with a RMSE loss of only 0.7 mm on NYU-Depth v2, as W_4A_4 sees instead significant degradation of the MRE, MNS, and MAE metrics (*e.g.*, 6.6 mm in RMSE). We select this model for the following analysis.

6.4. Qualitative Analysis

We now highlight the visual differences observed between different depth completion approaches. We consider the best competitor from Sec. 4.2, NLSPN, against our method in its float32 and MP W_4A_8 flavors. We report three samples from each dataset with both depth maps $\hat{D}$ and error maps $\hat{D} - D_{GT}$ in the same range to allow for a visual comparison. On NYU-Depth v2 NLSPN yields sharp edges with minimal amount of “mixed depth” pixels. However, the float32 and W_4A_8 models are as sharp, *e.g.*, on the wall discontinuities (middle row). The error patterns seen in all approaches on far planar regions (top row) are probably due to the acquisition of D_{GT} with a Kinect sensor, whose

accuracy decreases with the distance [53].

As for the higher-resolution SDS-ST1k dataset, we see that while depth map quality of our models is comparable to that of NLSPN, on several scenes the latter yields larger errors on planar surfaces (*e.g.*, the ground plane) than ours as shown in the corresponding error maps (top, middle row); our model also recovers more accurately some complex object details such as the cupboard (middle row), trash bin and shelves (bottom row).

7. Conclusion

We described a depth completion model based on a compact CNN with mixed-precision quantization. Our model performs within a few percent units of the state-of-the-art in terms of quality metrics on standard datasets such as NYU-Depth v2, but achieves faster GPU time at competitively small memory footprint and is suitable for real-time applications. A dedicated hardware implementation of our QNN will take full advantage of mixed-precision by fixed-point operations. We will investigate knowledge distillation with MP QAT to compress the network, as well as domain adaptation techniques [1, 31] to improve depth map quality on real RGB-ToF datasets.

References

- [1] Gianluca Agresti, Henrik Schafer, Piergiorgio Sartor, Yalcin Incesu, and Pietro Zanuttigh. Unsupervised Domain Adaptation of Deep Networks for ToF Depth Refinement. *IEEE Trans. Pattern Anal. Mach. Intell.*, 2021.
- [2] Yu Bai, Yu-Xiang Wang, and Edo Liberty. ProxQuant: Quantized Neural Networks via Proximal Operators. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [3] Jonathan T. Barron and Ben Poole. The Fast Bilateral Solver. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 617–632. Springer International Publishing, 2016.
- [4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [5] Dimitri P. Bertsekas. *Constrained optimization and Lagrange multiplier methods*. Academic Press, 2014.
- [6] Valerio Cambareri. SDS-ST1K Dataset. Technical report, Sony Depthsensing Solutions NV, May 2021.
- [7] Fabien Cardinaux, Stefan Uhlich, Kazuki Yoshiyama, Javier Alonso García, Lukas Mauch, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Iteratively Training Look-Up Tables for Network Quantization. *IEEE J. Sel. Top. Signal Process.*, 14(4):860–870, 2020.
- [8] Zhao Chen, Vijay Badrinarayanan, Gilad Drozdov, and Andrew Rabinovich. Estimating depth from rgb and sparse sensing. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 167–182, 2018.
- [9] Xinjing Cheng, Peng Wang, and Ruigang Yang. Learning depth with convolutional spatial propagation network. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(10):2361–2379, 2019.
- [10] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I.-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.
- [11] James Diebel and Sebastian Thrun. An Application of Markov Random Fields to Range Sensing. In Y. Weiss, B. Schölkopf, and J. Platt, editors, *Proc. Advances Neural Inf. Process. Syst. (NeurIPS)*, volume 18. MIT Press, 2005.
- [12] Abdelrahman Eldesokey, Michael Felsberg, Karl Holmquist, and Michael Persson. Uncertainty-Aware CNNs for Depth Completion: Uncertainty from Beginning to End. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.
- [13] A. Eldesokey, M. Felsberg, and F. Khan. Confidence Propagation through CNNs for Guided Sparse Depth Regression. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(10):2423–2436, Oct. 2020. Place: Los Alamitos, CA, USA Publisher: IEEE Computer Society.
- [14] Epic Games. Unreal Engine (v4.25).
- [15] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. Distance Transforms of Sampled Functions. *Theory of Computing*, 8(19):415–428, 2012.
- [16] Vitor Guizilini, Rares Ambrus, Wolfram Burgard, and Adrien Gaidon. Sparse auxiliary networks for unified monocular depth prediction and completion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 11078–11088, 2021.
- [17] Vitor Guizilini, Rares Ambrus, Sudeep Pillai, Allan Rantos, and Adrien Gaidon. 3D Packing for Self-Supervised Monocular Depth Estimation. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.
- [18] Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [19] Akio Hayakawa, Masato Ishii, Yoshiyuki Kobayashi, Akira Nakamura, Takuya Narihira, Yukio Obuchi, Andrew Shin, Takuya Yashima, and Kazuki Yoshiyama. Neural Network Libraries: A Deep Learning Framework Designed from Engineers’ Perspectives, 2021.
- [20] Mu Hu, Shuling Wang, Bin Li, Shiyu Ning, Li Fan, and Xiaojin Gong. PENet: Towards Precise and Efficient Image Guided Depth Completion. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 13656–13662, 2021.
- [21] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely Connected Convolutional Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017.
- [22] Saif Imran, Yunfei Long, Xiaoming Liu, and Daniel Morris. Depth Coefficients for Depth Completion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019.
- [23] Sambhav Jain, Albert Gural, Michael Wu, and Chris Dick. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. In I. Dhillon, D. Papailiopoulos, and V. Sze, editors, *Proceedings of Machine Learning and Systems*, volume 2, pages 112–128, 2020.
- [24] M. Jaritz, R. Charette, E. Wirbel, X. Perrotton, and F. Nashashibi. Sparse and Dense Data with CNNs: Depth Completion and Semantic Segmentation. In *Proc. IEEE Int. Conf. 3D Vis. (3DV)*, pages 52–60, 2018.
- [25] Alex Kendall, Matthew Grimes, and Roberto Cipolla. PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization. In *Proc. Int. Conf. Comput. Vis. (ICCV)*, pages 2938–2946, 2015.
- [26] Diederick P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [27] Jason Ku, Ali Harakeh, and Steven L. Waslander. In Defense of Classical Image Processing: Fast Depth Completion on the CPU. In *2018 15th Conference on Computer and Robot Vision (CRV)*, pages 16–22. IEEE, 2018.
- [28] A. Li, Z. Yuan, Y. Ling, W. Chi, S. Zhang, and C. Zhang. A Multi-Scale Guided Cascade Hourglass Network for Depth Completion. In *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, pages 32–40, 2020.
- [29] Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.
- [30] Zhi-Gang Liu and Matthew Mattina. Learning Low-precision Neural Networks without Straight-Through Estimator (STE). In *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, pages 3066–3072, 2019.

- [31] Adrian Lopez-Rodriguez, Benjamin Busam, and Krystian Mikolajczyk. Project to Adapt: Domain Adaptation for Depth Completion from Noisy and Sparse Sensor Data. In *Proc. Asian Conf. Comput. Vis. (ACCV)*, 2020.
- [32] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [33] Gregor Luetzenburg, Aart Kroon, and Anders A. Bjørk. Evaluation of the Apple iPhone 12 Pro LiDAR for an Application in Geosciences. *Scientific Reports*, 11(1), Nov. 2021.
- [34] Fangchang Ma, Guilherme Venturelli Cavalheiro, and Sertac Karaman. Self-Supervised Sparse-to-Dense: Self-Supervised Depth Completion from LiDAR and Monocular Camera. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 3288–3295, 2019.
- [35] Fangchang Ma and Sertac Karaman. Sparse-to-Dense: Depth Prediction from Sparse Depth Samples and a Single Image. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 4796–4803, 2018.
- [36] Miloš Nikolić, Ghouthi Boukli Hacene, Ciaran Bannon, Alberto Delmas Lascorz, Matthieu Courbariaux, Yoshua Bengio, Vincent Gripon, and Andreas Moshovos. Bitpruning: Learning bitlengths for aggressive and accurate quantization. *arXiv preprint arXiv:2002.03090*, 2020.
- [37] Jinsun Park, Kyungdon Joo, Zhe Hu, Chi-Kuei Liu, and In So Kweon. Non-local spatial propagation network for depth completion. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 120–136. Springer, 2020.
- [38] Vaishakh Patil, Wouter Van Gansbeke, Dengxin Dai, and Luc Van Gool. Don't Forget The Past: Recurrent Depth Estimation from Monocular Video. *IEEE Robot. Autom. Lett.*, 5(4):6813–6820, 2020.
- [39] Jiaxiong Qiu, Zhaopeng Cui, Yinda Zhang, Xingdi Zhang, Shuaicheng Liu, Bing Zeng, and Marc Pollefeys. DeepLiDAR: Deep Surface Normal Guided Depth Prediction for Outdoor Scene From Sparse LiDAR Data and Single Color Image. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019.
- [40] Chao Qu, Ty Nguyen, and Camillo Taylor. Depth completion via deep basis fitting. In *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, pages 71–80, 2020.
- [41] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. In Nassir Navab, Joachim Hornegger, William M. Wells, and Alejandro F. Frangi, editors, *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241, Cham, 2015. Springer International Publishing.
- [42] Shreyas S. Shivakumar, Ty Nguyen, Ian D. Miller, Steven W. Chen, Vijay Kumar, and Camillo J. Taylor. DFuseNet: Deep Fusion of RGB and Sparse Depth Information for Image Guided Dense Depth Completion. In *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, 2019.
- [43] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor Segmentation and Support Inference from RGBD Images. In Andrew Fitzgibbon, Svetlana Lazebnik, Pietro Perona, Yoichi Sato, and Cordelia Schmid, editors, *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 746–760, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [44] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, and others. Scalability in perception for autonomous driving: Waymo open dataset. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 2446–2454, 2020.
- [45] Lucas Teixeira, Martin R. Oswald, Marc Pollefeys, and Margarita Chli. Aerial Single-View Depth Completion with Image-Guided Uncertainty Estimation. *IEEE Robot. Autom. Lett.*, 5(2):1055–1062, 2020.
- [46] Stefan Uhlich, Lukas Mauch, Fabien Cardinaux, Kazuki Yoshiyama, Javier Alonso Garcia, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Mixed Precision DNNs: All you need is a good parametrization. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [47] J. Uhrig, N. Schneider, L. Schneider, U. Franke, T. Brox, and A. Geiger. Sparsity Invariant CNNs. In *Proc. IEEE Int. Conf. 3D Vis. (3DV)*, pages 11–20, 2017.
- [48] Wouter Van Gansbeke, Davy Neven, Bert De Brabandere, and Luc Van Gool. Sparse and Noisy LiDAR Completion with RGB Guidance and Uncertainty. In *2019 16th International Conference on Machine Vision Applications (MVA)*, pages 1–6. IEEE, 2019.
- [49] Felix Wimbauer, Nan Yang, Lukas von Stumberg, Niclas Zeller, and Daniel Cremers. MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 6108–6118, 2021.
- [50] Alex Wong, Xiaohan Fei, Stephanie Tsuei, and Stefano Soatto. Unsupervised Depth Completion From Visual Inertial Odometry. *IEEE Robot. Autom. Lett.*, 5(2):1899–1906, 2020.
- [51] Yan Xu, Xinge Zhu, Jianping Shi, Guofeng Zhang, Hujun Bao, and Hongsheng Li. Depth Completion From Sparse LiDAR Data With Depth-Normal Constraints. In *Proc. Int. Conf. Comput. Vis. (ICCV)*, 2019.
- [52] Yanchao Yang, Alex Wong, and Stefano Soatto. Dense Depth Posterior (DDP) From Single Image and Sparse Range. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019.
- [53] Pietro Zanuttigh, Giulio Marin, Carlo Dal Mutto, Fabio Dominio, Ludovico Minto, and Guido Maria Cortelazzo. *Time-of-Flight and Structured Light Depth Cameras*. Springer International Publishing, 2016.
- [54] Aojun Zhou, Anbang Yao, Yiwu Guo, Lin Xu, and Yurong Chen. Incremental Network Quantization: Towards Lossless CNNs with Low-Precision Weights. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.